



Proyecto Parte III- FINAL Generación de Ejecutable y Optimización

Instrucciones:

Este proyecto serán 10 puntos del examen final.

Podrá trabajar en grupos del proyecto anterior.

Los programas serán presentados el **viernes 15 y lunes 18 de mayo**, para su calificación práctica.

La documentación del proyecto se subirá de forma individual en formato PDF. Ultimo día **domingo 16 de mayo**. (screens del programa explicación de la realización del mismo, adjunto al trabajo previo)

OBJETIVO.

Transformar el Código Intermedio (TAC) en un archivo ejecutable funcional, aplicando técnicas de optimización para mejorar el rendimiento y la gestión de recursos.

INSTRUCCIONES:

Agregar al código del proyecto anterior lo que será el analizador generador de código intermedio, el código de maquina o ensamblador para generar el archivo .EXE que podrá ser corrido en base al código ingresado.

MENU PRINCIPAL

1. Ingreso de texto en el editor
2. Análisis léxico.
3. Análisis sintáctico en árbol
4. Tabla de símbolos
5. Analizador Semántico
6. Código intermedio.
7. Código optimizado.
8. Generador de código de máquina.
9. Correr archivo .EXE.
10. Salir.

- Analice código fuente de un lenguaje propio (con analizador léxico, sintáctico y semántico ya listos).
- Genere código intermedio optimizado.
- Genere código ensamblador real (por ejemplo, para NASM).
- Compile ese código ensamblador a .exe usando herramientas externas como NASM y GCC o MinGW.
- Herramientas posibles a utilizar:
 - Python 3.x (compilador hecho en Python)
 - NASM (ensamblador)
 - MinGW o GCC (para enlazar el archivo objeto .obj y generar el .exe)
 - (Opcional) Un sistema Linux o WSL en Windows, aunque también se puede en Windows nativo.

1. Fase de Optimización (Back-end)

Antes de emitir el código final, deben procesar su Representación Intermedia (RI).

Optimización Local: Realizar Constant Folding (evaluar $2+2$ en tiempo de compilación) y Constant Propagation.

Eliminación de Código Muerto: Quitar instrucciones que no afectan el resultado final o bloques inalcanzables.

Reducción de Fuerza: Reemplazar operaciones costosas (ej. $x * 2$ por $x << 1$).

2. Generación de Código Objeto (Ensamblador)

Deben mapear su RI a una arquitectura destino (Sugerencia: x86-64 para sistemas modernos).

Asignación de Registros: Implementar un algoritmo básico (como Linear Scan) o usar un enfoque basado en pila (más simple, pero menos eficiente).

- Generador de Ensamblador

Convertir el TAC optimizado a código x86 de 32 bits (para NASM).

Implementar reglas de optimización básicas (en optimizador.py)

Añadir soporte para más estructuras (ciclos, condicionales)

Crear su propio lenguaje mini tipo C.

Generar y ejecutar .exe para probar su compilación.



COMPILADORES

Manejo de Memoria: Definir el Stack Frame para funciones (Prólogo y Epílogo), gestionando variables locales y parámetros.

Llamadas al Sistema (Syscalls): Implementar funciones básicas de entrada/salida (como print o read) para interactuar con la consola.

3. Creación del Ejecutable

El compilador no solo debe imprimir texto, debe "construir".

Exportación: El programa debe generar un archivo con extensión .s o .asm.

Ensamble y Ligado: El compilador debe invocar automáticamente a herramientas externas (como nasm y gcc o ld) para transformar el código fuente en un binario ejecutable (.exe o .out).